

# GIORNO 06

---

**Funzioni: dare un nome a un pezzo di lavoro e non riscriverlo mai più**

# Cosa saprete fare stasera

Scrivere una funzione con parametri e valore di ritorno, e chiamarla più volte  
con argomenti diversi ottenendo i risultati attesi

Distinguere una funzione che restituisce da una che si limita a stampare,  
e scegliere la forma adatta al caso

Prendere un programma lungo già scritto ed estrarne almeno tre funzioni  
senza che l'output cambi di un carattere

Prevedere cosa succede quando si usa fuori una variabile nata dentro una funzione

Scrivere una funzione con parametri opzionali e chiamarla con argomenti nominati

Progettare la firma di una funzione a partire dalla descrizione del compito

È il cambio di forma più importante di tutto il blocco: oggi il codice smette  
di essere una sequenza!

# 1. RIPASSO DEI CICLI, E IL LIMITE CHE RESTA

---

Tre righe da correggere prima di aggiungere roba nuova

## 1.1 Ripasso attivo: l'accumulatore azzerato dentro il ciclo

```
# 1. Inizializzazione dell'accumulatore, FUORI dal ciclo: una volta sola.
totale = 0

# 2. Tre giri: numero vale 1, poi 2, poi 3. L'estremo destro e' escluso.
for numero in range(1, 4):
    # TRABOCCHETTO: indentata DENTRO il corpo, questa riga viene rieseguita a
    # ogni giro e butta il parziale. Non e' una variabile nuova: e' la stessa
    # di sopra, riportata a zero tre volte.
    totale = 0
    totale += numero

print(totale)

# 3

# Non 6: alla fine l'accumulatore contiene solo l'ultimo numero.
```

# 1.1 Ripasso attivo: range() con passo, e la sentinella

```
# Quante volte gira?
for indice in range(2, 8, 2):
    print(indice)

# 2 4 6 -> tre giri, non quattro: l'estremo destro e' escluso sempre.

# La sentinella e' "fine": il valore che fa uscire. La variabile parte da
# un valore qualsiasi, purché diverso dalla sentinella.
risposta = "no"
while risposta != "fine":
    # TRABOCCHETTO: il confronto e' fra stringhe ed e' sensibile alle
    # maiuscole. "Fine" != "fine" e' Vero, quindi il ciclo riparte. .strip()
    # toglie gli spazi ai bordi, non le maiuscole: per quelle serve .lower().
    risposta = input("Scrivi fine per uscire: ").strip()
```

## 1.2 Ripetere e nominare: due problemi diversi

| Problema                                             | Strumento               | Da quando                            |
|------------------------------------------------------|-------------------------|--------------------------------------|
| La stessa operazione, molte volte di seguito         | ciclo for / while       | Giorno 05                            |
| La stessa operazione, in punti diversi del programma | funzione                | oggi                                 |
| La stessa operazione, su una collezione di dati      | ciclo + strutture dati  | quando arriveranno le strutture dati |
| La stessa operazione, su dati che arrivano da fuori  | ciclo + funzioni + file | quando i dati arriveranno da un file |

*Ieri avete imparato a ripetere, oggi imparate a nominare. Il ciclo risolve "questa operazione va fatta cinquecento volte"; la funzione risolve "questa operazione va fatta in sette punti diversi del programma, e in tre di questi l'ho scritta leggermente diversa senza accorgermene". Le quattro righe non sono alternative: si sommano, e oggi mettete il pezzo che tiene su gli altri.*

## 1.3 La regola nuova: main() diventa il regista

Fino a ieri le soluzioni di questo corso stavano tutte dentro main().

Era voluto: non avevate ancora gli strumenti per fare altrimenti.

Da oggi vale il contrario.

Una soluzione tutta dentro main() è considerata un difetto,  
anche se funziona, anche se l'output è giusto.

Da oggi main() è il regista: chiama i pezzi nell'ordine e tiene i conti.

Il lavoro lo fanno le funzioni.

Vale per gli esercizi di oggi, per quelli di domani, per il progetto conclusivo  
del blocco e per i compiti a casa.

## 2. PERCHÉ SERVONO LE FUNZIONI

---

Codice monolitico e duplicazione: il momento preciso in cui si rompe

## 2.1 Il codice monolitico: il primo blocco, dal codice al prezzo

```
# 1. Accumulatore del totale, in virgola mobile perche' sono euro.
totale = 0.0

# 2. Primo blocco: dal codice articolo al prezzo. E' una tabella di
#     listino scritta come catena di elif, con il dato di listino
#     mescolato alla decisione.

codice = "MON"

if codice == "CUF":
    totale += 49.90

elif codice == "TAS":
    totale += 89.00

elif codice == "MON":
    totale += 219.00
```

*Un programma di venti righe si legge. Uno di cinquanta si legge con fatica. Uno di centoventi non si legge: si cerca dentro, come in un cassetto. Qui non c'è niente di sbagliato: gira e fa il suo lavoro.*

## 2.1 Il codice monolitico: il secondo blocco, e i numeri che si somigliano

```
# 3. Secondo blocco: dal totale alla percentuale di sconto. Argomento
#    diverso, stessa altezza nel file: niente dice che qui si cambia
#    lavoro.

if totale >= 200.00:
    sconto = totale * 10 / 100
elif totale >= 100.00:
    sconto = totale * 5 / 100
else:
    sconto = 0.0

# TRABOCCHETTO: 200.00 e 100.00 sono soglie, 219.00 e 49.90 sono prezzi.
#    Scritti cosi' si somigliano, e chi deve alzare la soglia dello sconto
#    li deve distinguere a occhio, riga per riga.

print(f"Totale: {totale - sconto:.2f}")

# Totale: 197.10
```

## 2.2 I tre costi del codice duplicato: ricerca, modifica, riuso

| Danno         | Come si manifesta                                               | Quanto costa            |
|---------------|-----------------------------------------------------------------|-------------------------|
| Non si trova  | "dove si calcola lo sconto?" richiede trenta secondi di lettura | tempo, ogni volta       |
| Non si cambia | la stessa regola scritta in tre punti: se ne aggiornano due     | un bug silenzioso       |
| Non si riusa  | serve la stessa cosa in un altro programma: si copia            | due copie che divergono |

*Il terzo è quello che oggi fa la differenza: il punteggio sulle parole spia che scrivete in 06.5 torna domani in 07.7, quando avrete le strutture dati, e poi in 11.5, quando i ticket arriveranno da un file. Analogia - la ricetta scritta a parte: nel ricettario di casa la besciamella ha una pagina sua, e lasagne, cannelloni e crespelle rimandano a pagina 42. Se ci vuole meno burro, correggete pagina 42 e avete corretto tre ricette.*

## 2.3 I tre segnali che un blocco va estratto in funzione

Non esiste un numero di righe oltre il quale bisogna spezzare:  
esistono tre segnali, e valgono più di qualunque numero.

1. Vi serve un commento per dire cosa fa un blocco.

Quel commento è il nome di una funzione che non avete ancora scritto.

2. State ricopiando.

Con il copia-incolla o riscrivendo a mano: è lo stesso.

3. Per capire una riga dovete risalire di quaranta righe  
a vedere che cosa contiene una variabile.

# 3. CHE COS'È UNA FUNZIONE

---

Un blocco di codice con un nome

## 3.1 Definizione di funzione: un blocco di codice con un nome

Una funzione è un pezzo di programma a cui avete dato un nome.

Da quel momento, per eseguire quel pezzo, basta scrivere il nome.

Entra qualcosa, esce qualcosa, e quello che succede dentro,  
mentre la usate, non vi interessa.

Analogia - il frullatore

ci mettete la frutta, premete il tasto, esce il frullato

non sapete come sono fatte le lame, non vi interessa la potenza del motore,

non avete mai aperto il coperchio inferiore

sapete due cose sole: cosa ci va dentro e cosa ne esce

Chi scrive una funzione lavora perché chi la usa possa continuare  
a non sapere cosa c'è dentro.

## 3.2 Anatomia di un def: nome, parentesi, due punti, corpo

```
# 1. DEFINIZIONE. Questa riga non esegue niente: registra il nome saluta e
#   gli attacca il blocco indentato che segue.
def saluta():
    print("Buongiorno")
```

```
# 2. CHIAMATA: qui il blocco viene eseguito, una volta per ogni riga.
# TRABOCCHETTO: le parentesi non sono decorazione. Senza, il nome saluta
#   vale la funzione stessa e non la esegue: ne riparlamo in E5.
saluta()
saluta()

# Buongiorno

# Buongiorno
```

*Due righe vuote fra la definizione e quello che segue: è PEP 8, ed è quello che rende visibile a colpo d'occhio dove finisce una funzione.*

## 3.2 Anatomia di un def: i cinque pezzi, uno per uno

| Pezzo              | Che cos'è                                                         |
|--------------------|-------------------------------------------------------------------|
| def                | la parola che dice a Python "sto definendo una funzione"          |
| saluta             | il nome che le date, con le stesse regole dei nomi di variabile   |
| ()                 | le parentesi: qui dentro andranno i parametri, per ora sono vuote |
| :                  | i due punti, come nell'if e nel for: annunciano un blocco         |
| il corpo indentato | quello che la funzione fa, quattro spazi più a destra             |

*E poi, staccate di due righe vuote, le chiamate: `saluta()`. Con le parentesi, sempre. Il rientro è la stessa cosa imparata al Giorno 04 con l'if: il blocco finisce dove finisce il rientro, e non serve nessuna parentesi graffa a dirlo.*

## 3.3 Definizione e chiamata: definire non esegue

```
# Python legge questa riga, registra il nome saluta e tira dritto SENZA
# entrare nel corpo.

def saluta():

# TRABOCCHETTO: questa print e' scritta, e' corretta, ed e' dentro il
# corpo. Non viene eseguita perche' nessuno ha mai chiamato saluta().
    print("Buongiorno")


# A colonna 0: non appartiene alla funzione, ed e' l'unica riga che gira.
print("Fine del programma")


# Fine del programma
```

*Il saluto non compare. Definire una funzione non la esegue: è come scrivere una ricetta, e scriverla non prepara la cena. È il primo punto che sfugge, e sfugge a tutti.*

## 3.4 Ordine nel file: NameError se la chiamata precede il def

```
print(prezzo_cuffie())  
# TRABOCCHETTO: questa e' la riga 1, e quando Python la esegue il nome  
# prezzo_cuffie non esiste ancora. Il nome nasce nell'istante in cui il  
# def viene letto, e il def sta sotto: il traceback dice solo questo.
```

```
# La definizione e' corretta. E' solo scritta troppo in basso.  
def prezzo_cuffie():  
    return 49.90
```

```
Traceback (most recent call last):  
  File "prova.py", line 1, in <module>  
    print(prezzo_cuffie())  
    ^^^^^^^^^^^^^^^
```

```
NameError: name 'prezzo_cuffie' is not defined
```

*Python legge il file dall'alto verso il basso: quando incontra una chiamata, il nome deve già esistere. Regola pratica che chiude il problema per sempre: tutte le funzioni in alto, main() sotto di loro, la chiamata a main() in fondo dentro la guard.*

## 3.5 def main() e la guardia if \_\_name\_\_ == "\_\_main\_\_"

```
# 1. main e' una funzione come tutte le altre: il nome e' una convenzione,  
#     non una parola chiave, e Python non la tratta in modo speciale.
```

```
def main():  
    """Fa il lavoro del programma."""  
    print("NovaStore")
```

```
# 2. __name__ e' un nome che Python riempie da solo, e vale "__main__" se  
#     questo file e' stato lanciato direttamente: la riga sotto e' quindi  
#     una condizione ordinaria, con dentro una chiamata ordinaria.  
# TRABOCCHETTO: senza questa chiamata finale il programma non stampa nulla  
# e non da' errore. Ha definito una funzione e ha chiuso.  
if __name__ == "__main__":
```

```
    main()
```

*Dal primo giorno scrivete questo, e la risposta era "il perché completo lo capirete quando arriveremo alle funzioni". Ci siamo arrivati. Per voi quella riga vuol dire "adesso parti"; in azienda è quello che separa un file utile da un programma che riparte quando non deve.*

## 3.6 PEP 8: due righe vuote fra le funzioni, una dentro

Fra una funzione e l'altra si lasciano DUE righe vuote.

È PEP 8, la convenzione di stile di Python.

Dentro una funzione, per separare due blocchi logici,  
si lascia UNA riga vuota.

Due righe per separare le funzioni, una per separare i pensieri.

Non è pignoleria: è la cosa che rende leggibile a colpo d'occhio  
dove finisce una funzione e dove ne comincia un'altra.

In un file di trenta funzioni è l'unica cosa che vi dice  
dove siete senza leggere il codice.

# 4. PARAMETRI E ARGOMENTI

---

Lo stesso lavoro su dati diversi

## 4.1 Il parametro: la casella che la funzione riceve

```
# 1. nome e' il PARAMETRO: una casella vuota che esiste solo dentro questa  
#     funzione e che si riempie al momento della chiamata.
```

```
def saluta(nome):  
    print(f"Buongiorno {nome}")
```

```
# 2. Le due stringhe sono gli ARGOMENTI: il valore che riempie la casella.  
# TRABOCCHETTO: il corpo e' stato scritto una volta sola. Le due righe qui  
#     sotto non sono due copie del codice: sono due esecuzioni dello stesso  
#     codice, con un dato diverso dentro.
```

```
saluta("Marta")  
  
saluta("Studio Legale Conti")
```

```
# Buongiorno Marta
```

```
# Buongiorno Studio Legale Conti
```

*Analogia - il modulo prestampato. Le caselle vuote (cognome, data, importo) sono i parametri; quello che ci scrivete sopra con la penna, ogni volta diverso, sono gli argomenti. Lo stesso modulo serve per mille pratiche, e nessuno si sogna di ristamparlo con i dati già dentro.*

## 4.2 Parametro e argomento: le due parole, e dove stanno

| Parola    | Dove sta                    | Esempio                                    |
|-----------|-----------------------------|--------------------------------------------|
| parametro | nella definizione, dopo def | <code>def saluta(nome): -&gt; nome</code>  |
| argomento | nella chiamata              | <code>saluta("Marta") -&gt; "Marta"</code> |

*Serve saperlo perché i messaggi d'errore di Python usano queste parole e non le usano a caso. Il parametro è il nome della casella, l'argomento è il valore che ci finisce dentro: due cose diverse, e Python le nomina separatamente quando qualcosa non torna.*

## 4.2 Parametro e argomento: i due termini nel messaggio d'errore

```
def calcola_iva(imponibile):  
    return imponibile * 0.22
```

```
print(calcola_iva())  
# TRABOCCHETTO: la riga qui sopra e' la riga 5, e le parentesi sono vuote.  
#   imponibile non ha un valore di partenza, quindi e' obbligatorio:  
#   Python non inventa uno zero per voi, si ferma e dice quale nome manca.
```

```
Traceback (most recent call last):  
  File "prova.py", line 5, in <module>  
    print(calcola_iva())  
          ^^^^^^^^^^^^^^^  
TypeError: calcola_iva() missing 1 required positional argument: 'imponibile'
```

*Tradotto: manca un argomento obbligatorio e posizionale, quello che nella definizione si chiama imponibile. Il messaggio vi dice il nome del parametro mancante, il che rende la correzione immediata. Anche il contrario dà errore, e anche lì il messaggio è preciso: takes 1 positional argument but 2 were given.*

## 4.3 Più parametri: l'ordine posizionale conta

```
# Due parametri separati da virgola. L'ordine in cui sono scritti qui e'  
# l'unica cosa che lega un argomento alla sua casella: si chiama passaggio  
# POSIZIONALE, perche' conta la posizione e non il nome.  
def rata_mensile(capitale, mesi):  
    return capitale / mesi
```

```
# Chiamata corretta: 2400 euro finiscono in capitale, 12 in mesi.  
print(rata_mensile(2400.00, 12))  
  
# TRABOCCHETTO: qui gli argomenti sono invertiti. Python li accetta senza  
# fiatare, perche' entrambi sono numeri e la divisione si puo' fare: 12  
# diviso 2400 fa 0.005. Nessun errore, un risultato senza senso.  
print(rata_mensile(12, 2400.00))
```

```
# 200.0
```

```
# 0.005
```

*Un numero giusto e un numero assurdo, e Python non ha modo di sapere quale dei due volevate. Con una moltiplicazione l'inversione non si noterebbe, perché il prodotto è lo stesso: con una divisione il caso finisce. È il difetto numero 3 di 06.7, ed è il motivo per cui al capitolo 9 impareremo a scrivere gli argomenti con il loro nome.*

## 4.4 Il parametro è una variabile locale della funzione

Il nome del parametro esiste solo dentro la funzione.

Dopo aver chiamato `saluta("Marta")`, se provate a scrivere `print(nome)` fuori,

Python risponde: `NameError: name 'nome' is not defined`

Se il nome del parametro uscisse fuori, due funzioni scritte da due persone diverse si ruberebbero i nomi a vicenda.

# 5. RETURN: FAR USCIRE IL RISULTATO

---

La distinzione più importante della giornata

## 5.1 print contro return: a schermo sembrano identiche

```
# 1. Questa funzione STAMPA: manda il risultato allo standard output,  
#     cioe' alla finestra. Al chiamante non torna indietro niente.  
def iva_stampata(imponibile):  
    print(imponibile * 0.22)  
  
# 2. Questa funzione RESTITUISCE: il risultato torna al punto della  
#     chiamata, e sulla finestra non compare niente da sola.  
def iva_restituita(imponibile):  
    return imponibile * 0.22  
  
# 3. Stesso 22.0 a schermo per due strade opposte: la prima stampa da  
#     dentro, la seconda viene stampata da fuori.  
# TRABOCCHETTO: non giudicate una funzione da quello che vedete a schermo.  
#     Conta che cosa torna al chiamante, e a schermo non si vede.  
iva_stampata(100.00)  
print(iva_restituita(100.00))  
  
# 22.0  
# 22.0
```

## 5.1 print contro return: provate a USARE il risultato

```
print(100.00 + iva_restituita(100.00))
print(100.00 + iva_stampata(100.00))
```

```
# La riga 9 somma 100.00 al valore restituito: 122.0, tutto regolare.
# TRABOCCHETTO: la riga 10 si ferma, ma PRIMA stampa 22.0. Quel 22.0 non
#   e' la somma: e' la print interna della funzione, che ha fatto il suo
#   lavoro. Al chiamante e' tornato None, e float + None non esiste.
```

```
122.0
22.0
```

```
Traceback (most recent call last):
```

```
  File "prova.py", line 10, in <module>
    print(100.00 + iva_stampata(100.00))
            ~~~~~^~~~~~
```

```
TypeError: unsupported operand type(s) for +: 'float' and 'NoneType'
```

*La differenza non si vede guardando il risultato: si vede provando a usarlo. La seconda riga stampa 22.0 - la funzione ha fatto il suo lavoro - e poi si ferma, perché quello che ha restituito al chiamante è None, e None non si somma a niente.*

## 5.1 print contro return: la lavagna e il foglio

Analogia - restituire contro affiggere.

Una funzione che STAMPA è come un collega che scrive il risultato sulla lavagna della sala riunioni: lo vedete, lo potete leggere ad alta voce, ma non lo avete in mano.

Una funzione che RESTITUISCE ve lo passa su un foglio: lo potete sommare, confrontare, mettere in una busta, passare a un'altra persona.

Sulla lavagna ci si scrive alla fine; il foglio si passa durante il lavoro.

Se stasera uscite avendo capito solo questa, la giornata è riuscita.

## 5.2 Scegliere fra return e print: tabella di decisione

| Se la funzione...                        | allora...            | esempio di oggi                     |
|------------------------------------------|----------------------|-------------------------------------|
| produce un valore che verrà riusato      | return               | calcola_iva, classifica_ticket      |
| decide qualcosa e la decisione va usata  | return               | percentuale_sconto, verdetto_ticket |
| serve solo a mostrare qualcosa a schermo | print, niente return | stampa_bolla, stampa_riepilogo      |
| fa tutte e due le cose                   | sono due funzioni    | vedi capitolo 12                    |

*La regola che vale il novanta per cento dei casi: chi calcola non stampa, chi stampa non calcola. Notate i due nomi sulla prima e sulla seconda riga: `classifica_ticket()` risponde "di che cosa parla questo ticket" e restituisce una fra cinque categorie, `verdetto_ticket()` risponde "entra in coda oppure no". Sono due domande diverse sullo stesso testo, e restano due funzioni separate.*

## 5.3 return interrompe la funzione: uscita immediata

```
def totale_rate(rata, mesi):  
    # Accumulatore prima del ciclo, ciclo scritto per girare dodici volte:  
    # fin qui e' il ripasso di ieri, ed e' tutto corretto.  
    totale = 0.0  
    for numero_rate in range(mesi):  
        totale += rata  
    # TRABOCCHETTO: contate i rientri. Questo return sta a otto spazi, cioe'  
    #   DENTRO il corpo del for. Viene eseguito al primo giro e fa uscire  
    #   dalla funzione, non dal ciclo: il secondo giro non arriva mai.  
    return totale  
  
print(totale_rate(200.00, 12))
```

*Dodici rate da duecento euro non fanno duecento euro. Il ciclo c'è, gira una volta e non serve a niente. Nessun errore, nessun avviso.*

```
# 200.0
```

## 5.3 return interrompe la funzione: uno scarto di quattro spazi

SBAGLIATO

```
def totale_rate(rata, mesi):
    totale = 0.0
    for numero_rata in range(mesi):
        totale += rata
    return totale
```

```
print(totale_rate(200.00, 12))
```

```
# 200.0
```

il ciclo gira una volta sola  
e non serve a niente

GIUSTO

```
def totale_rate(rata, mesi):
    totale = 0.0
    for numero_rata in range(mesi):
        totale += rata
    return totale
```

```
print(totale_rate(200.00, 12))
```

```
# 2400.0
```

il ciclo gira dodici volte  
e poi si esce con il totale

*La correzione è spostare il return di quattro spazi a sinistra, fuori dal for e allineato con esso. Il difetto era di indentazione: il ciclo e il calcolo erano corretti. È il difetto numero 6 di 06.7, e a schermo condiviso quattro spazi sono quasi invisibili.*

## 5.4 return dopo un while: la funzione che valida un dato

```
def chiedi_intero(messaggio, minimo, massimo):  
    """Chiede un intero finché non ne arriva uno valido nell'intervallo."""  
  
    # 1. Variabili di lavoro locali, inizializzate PRIMA del ciclo: il valore  
    # raccolto e la bandiera che dice se siamo a posto.  
    valore = minimo  
    valido = False  
    while not valido:  
  
    # 2. Il dato letto e' sempre testo: prima la forma, poi il merito.  
    testo = input(messaggio).strip()  
  
    if not testo.isdecimal():  
        print("[ERRORE] Servono solo cifre.")  
  
    elif int(testo) < minimo or int(testo) > massimo:  
        print(f"[ERRORE] Serve un numero fra {minimo} e {massimo}.")  
  
    else:  
  
    # 3. Solo qui il dato e' buono: da str a int, e si alza la bandiera.  
    valore = int(testo)  
    valido = True  
  
    # TRABOCCHETTO: questo return e' a quattro spazi, allineato con il while,  
    # quindi e' FUORI dal ciclo. Spostatelo dentro e la funzione  
    # restituisce al primo giro, dato buono o no.  
  
    return valore
```

*Il return ci arriva solo quando valido è diventato True, cioè quando il dato è buono. Tre parametri e nessuna idea di che cosa serva il numero: è proprio per questo che si riusa ovunque. È l'esercizio 06.3, e la riuserete in casa\_06.3 e nel mini-progetto.*

## 5.5 Da provare subito: la funzione senza return

```
def raddoppia(numero):  
    # TRABOCCHETTO: la moltiplicazione viene eseguita davvero, e il risultato  
    # viene buttato. Senza return non esce niente: esce None.  
    numero * 2
```

```
print(raddoppia(21))
```

```
# None
```

```
def raddoppia(numero):  
    return numero * 2
```

```
print(raddoppia(21))
```

```
# 42
```

*Tre righe nell'editor, poi eseguite. Python non vi avvisa in nessun modo: la moltiplicazione viene fatta e il risultato buttato. Aggiungete il return e rieseguite. Scrivete in chat cosa vi esce nelle due versioni.*

# 6. LA FUNZIONE SENZA RETURN E IL VALORE NONE

---

None non è un errore, finché qualcuno non prova a usarlo

## 6.1 None: il valore restituito da una funzione senza return

```
# Funzione di sola stampa: lavora sullo standard output e non ha niente da
# consegnare al chiamante. Nessun return, ed e' corretto cosi'.

def stampa_riga():
    print("-" * 20)

# Il valore restituito si puo' catturare lo stesso: e' None.
risultato = stampa_riga()

# TRABOCCHETTO: type() dice che None NON e' la parola "None" e non e'
# zero. E' un valore di tipo NoneType, e su di lui non si fanno conti.
print(risultato, type(risultato))

# -----

# None <class 'NoneType'>
```

*Una funzione senza return restituisce None. Non è un guasto: è il valore che in Python significa "niente". Qui la funzione ha stampato la riga, che era il suo lavoro, e ha restituito None, perché non aveva niente da restituire. Tutto corretto.*

## 6.2 TypeError su NoneType: il sintomo lontano dalla causa

12.0

Traceback (most recent call last):

File "prova.py", line 6, in <module>

totale = 2400.00 + calcola\_commissione(2400.00)

~~~~~^~~~~~

TypeError: unsupported operand type(s) for +: 'float' and 'NoneType'

Guardate l'ordine delle righe:

12.0                   <- la funzione ha stampato, e il numero e' giusto

Traceback            <- poi il programma si e' fermato

Chi legge solo il traceback pensa che il calcolo non sia stato fatto.

Il calcolo e' stato fatto benissimo: e' il risultato che non e' tornato.

*È l'errore più frequente della giornata, e succede a tutti, oggi e fra dieci anni. Il riflesso da costruire: se leggete NoneType, cercate una funzione senza return.*

## 6.3 Tre sintomi che portano a un return mancante

| Sintomo                                                      | Causa quasi certa                                   |
|--------------------------------------------------------------|-----------------------------------------------------|
| TypeError: ... 'NoneType'                                    | una funzione che doveva restituire non ha il return |
| la funzione stampa il numero giusto ma il totale è sbagliato | stesso problema, visto da un'altra angolazione      |
| AttributeError: 'NoneType' object has no attribute 'upper'   | avete chiamato un metodo di stringa su un None      |

*Fra sei mesi questo capitolo non ve lo ricorderete, ma il traceback ve lo ritroverete davanti. Tre sintomi diversi, e portano tutti e tre allo stesso posto: una funzione che ha calcolato e non ha restituito.*

## 6.4 return dentro un if: None restituito solo a volte

```
def sconto_applicabile(totale):  
    # TRABOCCHETTO: questo if non ha else, e il return sta solo su un ramo.  
    # Quando la condizione e' falsa la funzione arriva in fondo e  
    # restituisce None: un return dentro un if e' un return che qualche  
    # volta non c'e'.  
    if totale >= 100.00:  
        return 10  
  
print(sconto_applicabile(150.00))  
  
# Ramo falso: torna None. Qui nessun errore, perche' print() stampa  
# qualunque cosa. L'errore arrivera' dove quel None viene sottratto.  
print(sconto_applicabile(50.00))  
  
# 10  
  
# None
```

*Nessun errore. Poi qualcuno scrive totale - sconto\_applicabile(50.00) e il programma si ferma quaranta righe più in là, in un punto che con lo sconto non c'entra niente. È il caso peggiore: quello in cui None non dà errore subito.*

## 6.4 La forma robusta: un valore di partenza, un return solo

```
def sconto_applicabile(totale):  
    """Restituisce la percentuale di sconto che spetta a un totale."""  
    # 1. Valore di partenza sensato: lo sconto di chi non ne ha diritto e'  
    #     zero, non "niente". La variabile esiste su tutti i percorsi.  
    percentuale = 0  
    if totale >= 100.00:  
        percentuale = 10  
    # 2. Un return solo, in fondo, fuori da ogni if: raggiungibile sempre.  
    #     E' la forma che rende impossibile il None.  
    return percentuale  
  
print(sconto_applicabile(150.00), sconto_applicabile(50.00))  
  
# 10 0
```

*Una variabile con un valore di partenza sensato, gli if che eventualmente la cambiano, un return solo alla fine: è la forma di tutte le soluzioni di oggi. Chi già programma sa che si può uscire con più return e spesso si legge meglio: è vero, e nei vostri esercizi nessuno ve lo vieta.*

# 7. COMPOSIZIONE: UNA FUNZIONE CHE NE CHIAMA UN'ALTRA

---

I pezzi si compongono

# 7.1 Composizione: una funzione ne chiama un'altra

```
# Costante in cima al file, in maiuscolo: un numero che ha un nome smette
# di essere un numero magico sparso per il programma.

ALIQUOTA_IVA = 0.22
```

```
# 1. Funzione di base: sa una cosa sola, e la sa bene.
```

```
def calcola_iva(imponibile):

    """Restituisce l'IVA dovuta su un imponibile."""

    return imponibile * ALIQUOTA_IVA
```

```
# 2. Funzione che compone: dentro il proprio return chiama la precedente.
```

```
def prezzo_finale(imponibile):

    """Restituisce l'imponibile più la sua IVA."""

# TRABOCCHETTO: calcola_iva(imponibile) qui NON stampa niente. Sta al
# posto del suo valore di ritorno, come ci fosse scritto 22.0, ed e'
# possibile solo perche' quella funzione restituisce invece di stampare.

    return imponibile + calcola_iva(imponibile)
```

```
print(f"{prezzo_finale(100.00):.2f}")
```

```
print(f"{prezzo_finale(49.90):.2f}")
```

## 7.2 Perché la composizione conviene: leggibilità per livelli

```
def costo_collo(zona, peso, urgente):  
    """Restituisce il costo di un collo, urgenza compresa."""  
  
    # Due chiamate dentro una somma: questa riga dice CHE COSA si calcola, il  
    # COME sta dentro le due funzioni, che qui non si aprono.  
  
    base = tariffa_di_zona(zona) + peso_al_chilo(peso)  
  
    # urgente e' gia' un booleano: scrivere if urgente == True: e' la stessa  
    # cosa, con tre parole in piu'.  
  
    if urgente:  
        base = base + supplemento_urgenza(base)  
  
    return base  
  
  
# Si legge ad alta voce come una frase in italiano:  
# il costo e' la tariffa di zona piu' il peso al chilo;  
# se e' urgente si aggiunge il supplemento.
```

## 7.3 Profondità delle chiamate: tre livelli, uno sopra l'altro

```
# LIVELLO 1: conta, e basta. Non divide, non stampa, non formatta.

def conta_vocali(testo):

    """Conta le vocali di un testo."""

    vocali = 0

# .lower() restituisce una copia in minuscolo: le stringhe sono
# immutabili, quindi testo non viene modificato.

    for carattere in testo.lower():

        if carattere in "aeiou":

            vocali += 1

    return vocali


# LIVELLO 2: usa il livello 1 e ci costruisce sopra una percentuale.

def densita_vocali(testo):

    """Restituisce la percentuale di vocali sul totale dei caratteri."""

# TRABOCCHETTO: len(testo) e' il denominatore e nessuno ha controllato che
# non sia zero. Su un testo vuoto questa riga esplode, e l'errore verra'
# attribuito a chi ha chiamato densita_vocali, non a chi l'ha scritta.

    return conta_vocali(testo) / len(testo) * 100


# LIVELLO 3, il chiamante: qui si decide solo come mostrare il risultato.

print(f"{densita_vocali('Poliambulatorio Aurora'):.1f}%")
```

## 7.3 Fin dove si scende, e perché la ricorsione resta fuori

Nessun limite pratico alla profondità delle chiamate,  
ma un limite di buon senso: oltre tre livelli si fa fatica a seguire.

Una funzione può anche chiamare se stessa.

Si chiama ricorsione e qui non la vediamo.

Se ci provate e il programma si pianta, manca la condizione di uscita.

Il criterio per fermarsi non è il numero di livelli: è la frase.

Se la funzione di sopra si legge ad alta voce come una frase in italiano,  
la profondità va bene. Se per capirla dovete aprire quella sotto, no.

## 7.4 Traceback a più livelli: ZeroDivisionError dentro una chiamata

```
print(densita_vocali(""))
```

Traceback (most recent call last):

```
File "prova.py", line 5, in <module>
```

```
    print(densita_vocali(""))
```

```
    ^^^^^^^^^^^^^^^^^^^^^
```

```
File "prova.py", line 2, in densita_vocali
```

```
    return conta_vocali(testo) / len(testo) * 100
```

```
    ~~~~~^~~~~~
```

ZeroDivisionError: division by zero

Due righe File, non una. Si legge dal basso:

l'ultima riga -> che cosa e' successo

quella sopra -> in quale funzione

quella ancora sopra -> chi l'aveva chiamata

*Con le funzioni il traceback è lo strumento di diagnosi principale: è la mappa del percorso. La correzione è un if sul denominatore e costa una riga - prima di dividere si guarda il divisore - e va scritta adesso, non il giorno in cui il file del fornitore arriva con una descrizione vuota.*

# 8. VALORI DI DEFAULT

---

Il parametro che si può omettere

## 8.1 Valore di default: il parametro che si può omettere

```
PERCENTUALE_PREDEFINITA = 10

# prezzo e' obbligatorio: nessun valore dopo il nome. percentuale ha un
# VALORE DI DEFAULT, quindi diventa facoltativo.

def applica_sconto(prezzo, percentuale=PERCENTUALE_PREDEFINITA):
    """Restituisce il prezzo scontato della percentuale indicata."""
    return prezzo - prezzo * percentuale / 100

# Un argomento solo: percentuale non e' "vuota", vale 10.
print(f"{applica_sconto(49.90):.2f}")

# TRABOCCHETTO: l'uguale nella riga del def non e' un'assegnazione
# eseguita a ogni chiamata, ed e' cosa diversa dall'uguale che vedrete
# al capitolo 9 dentro la chiamata. Stesso simbolo, due significati.
print(f"{applica_sconto(49.90, 20):.2f}")

# 44.91
# 39.92
```

*La prima chiamata non passa la percentuale: vale 10. La seconda la passa: vale 20.*

## 8.2 Ordine dei parametri: i default vanno per ultimi

```
def applica_sconto(percentuale=10, prezzo):  
    return prezzo - prezzo * percentuale / 100  
# TRABOCCHETTO: il corpo e' corretto e i nomi sono buoni. E' la riga 1 a  
# essere illegale, ed e' un errore di sintassi: il programma non parte  
# nemmeno. Con questa firma, applica_sconto(50) sarebbe ambigua.
```

File "prova.py", line 1

```
def applica_sconto(percentuale=10, prezzo):  
    ^^^^^
```

SyntaxError: parameter without a default follows parameter with a default

*Un parametro senza default segue un parametro con default: non è una preferenza, è un vincolo del linguaggio. Il messaggio è insolitamente chiaro, e ha ragione: se accettasse questa forma, Python non avrebbe modo di capire cosa significa applica\_sconto(50).*

## 8.3 Quando mettere un default: quattro situazioni tipiche

| Situazione                                                      | Default?                            |
|-----------------------------------------------------------------|-------------------------------------|
| Esiste un valore che va bene nel 90% dei casi                   | sì, ed è il caso di scuola          |
| Il valore cambia sempre                                         | no: rendetelo obbligatorio          |
| Il valore dipende dal contesto e dimenticarlo è pericoloso      | no: meglio l'errore che il silenzio |
| Serve aggiungere un parametro senza rompere il codice esistente | sì: è il motivo più forte           |

*Il default non è una comodità di chi scrive la funzione: è una promessa a chi la chiamerà. L'ultima riga merita una riga in più: se aggiungete un parametro senza default a una funzione già usata in dodici punti, quei dodici punti smettono di funzionare tutti insieme. Con un default continuano esattamente come prima, ed è quello che si fa in casa\_06.4.*

## 8.4 Il default che nasconde un errore: valore comodo, non giusto

```
def costo_spedizione(peso, tariffa=0.0):  
    """Restituisce il costo di una spedizione."""  
  
    # TRABOCCHETTO: zero non e' una tariffa, e' l'assenza di una tariffa. Come  
    # default trasforma una dimenticanza in un numero plausibile: 12.0 * 0.0  
    # fa 0.0, e nessun controllo lo intercettera' piu'.  
    return peso * tariffa  
  
# Chiamata incompleta. Senza quel default qui ci sarebbe un TypeError.  
print(costo_spedizione(12.0))  
  
# 0.0
```

*Nessun errore, e la spedizione risulta gratis. Senza default Python si sarebbe fermato subito, e il problema sarebbe emerso in aula invece che in fattura. Il criterio: un default va messo quando c'è un valore giusto, non quando c'è un valore comodo.*

## 8.5 Il default si valuta alla riga def, una volta sola

Il valore di default viene calcolato quando Python legge la riga def, non a ogni chiamata.

Se scrivete `def verdetto(punteggio, soglia=SOGLIA):`  
e più tardi cambiate `SOGLIA`, il default resta quello di prima.

Per noi la conseguenza è una sola, ed è tranquillizzante:  
le costanti in cima al file non si cambiano mentre il programma gira,  
quindi non capita mai.

Per chi vuole andare oltre: il tranello vero nasce quando il valore di default è un oggetto modificabile. Ha un nome (mutable default argument) e lo incontrerete più avanti nella vostra carriera.

# 9. ARGOMENTI NOMINATI

---

**Chiamare dicendo i nomi**

## 9.1 Argomento nominato: nome=valore nella chiamata

```
def applica_sconto(prezzo, percentuale=10):  
    return prezzo - prezzo * percentuale / 100  
  
# 1. Tutto POSIZIONALE: 219.00 va in prezzo perche' scritto per primo.  
print(f"{applica_sconto(219.00, 50):.2f}")  
  
# 2. Tutto NOMINATO: ogni valore dichiara la casella in cui va.  
print(f"{applica_sconto(prezzo=219.00, percentuale=50):.2f}")  
  
# TRABOCCHETTO: qui l'ordine e' invertito e il risultato e' identico. Con  
#   i nomi la posizione smette di contare, ed e' l'unica protezione  
#   contro l'inversione silenziosa vista in 4.3.  
print(f"{applica_sconto(percentuale=50, prezzo=219.00):.2f}")  
  
# 109.50  
# 109.50  
# 109.50
```

## 9.2 A cosa servono i nomi: l'inversione silenziosa

Servono a due cose, e tutte e due riguardano chi leggerà il codice fra sei mesi (probabilmente voi).

Primo: proteggono dall'ordine sbagliato.

`rata_mensile(12, 2400.00)` restituiva 0.005 e nessuno se ne accorgeva;  
`rata_mensile(capitale=2400.00, mesi=12)` restituisce 200.0 comunque  
li scriviate, e non si può scambiare per sbaglio.

Secondo: rendono leggibile la chiamata.

Ed è il motivo per cui gli argomenti nominati vanno venduti come una protezione, non come una finezza di stile.

## 9.2 A cosa servono i nomi: sei argomenti che nessuno sa leggere

```
# TRABOCCHETTO: che cosa sono 14, False, 500.00 e 3? Per saperlo bisogna
# aprire la definizione della funzione. Sei argomenti posizionali sono
# sei occasioni di invertirne due senza che niente lo segnali.
stampa_riga_controllo("Bonifico A", 250.00, 14, False, 500.00, 3)
```

```
# Gli ultimi due sono nominati e la chiamata si legge da sola. La riga si
# spezza dopo una virgola, allineata sotto la parentesi: e' PEP 8.
stampa_riga_controllo("Bonifico A", 250.00, 14, False,
                      soglia_importo=500.00,
                      soglia_blocco=3)
```

*La prima è più corta. La seconda si capisce senza andare a rileggere la definizione della funzione. Nel codice che qualcuno dovrà mantenere, la seconda vince sempre.*

## 9.3 La regola: prima i posizionali, poi i nominati

```
print(appllica_sconto(percentuale=25, 89.00))
```

```
File "prova.py", line 5
```

```
    print(appllica_sconto(percentuale=25, 89.00))
                                   ^^^^^
```

```
SyntaxError: positional argument follows keyword argument
```

Python non arriva nemmeno a eseguire: si ferma alla lettura del file.

*Una volta cominciato a nominare, si continua a nominare. Se ci pensate ha senso: dopo percentuale=25, il valore 89.00 in che posizione starebbe?*

## 9.4 Saltare un parametro in mezzo lasciandolo al default

```
# Un obbligatorio e due con default: la firma dichiara che il caso normale
# e' "fra quattro e otto parole", e che si puo' derogare. Nel corpo, la
# forma robusta di 6.4: valore di partenza, gli if lo cambiano, un return.

def valuta_descrizione(parole, minimo=4, massimo=8):

    """Restituisce l'esito del controllo di lunghezza."""

    esito = "[OK] descrizione nella norma"

    if parole < minimo:

        esito = "[!] troppo corta: aggiungere dettagli"

    elif parole > massimo:

        esito = "[!] troppo lunga: accorciare"

    return esito


# Entrambi i default in vigore: nove parole superano il massimo di otto.
print(valuta_descrizione(9))

# TRABOCCHETTO: qui si cambia il TERZO parametro lasciando il secondo al
# default. Posizionalmente non si puo' fare: valuta_descrizione(9, 12)
# metterebbe 12 in minimo, cioe' nella casella sbagliata.
print(valuta_descrizione(9, massimo=12))


# [!] troppo lunga: accorciare
# [OK] descrizione nella norma
```

*È qui che gli argomenti nominati diventano indispensabili: senza i nomi bisognerebbe ripetere anche il minimo, e ripeterlo significa poterlo sbagliare. È NovaStore, ed è l'esercizio 06.6.*

## 9.5 Tabella di decisione: posizionale, nominato, misto

| Come chiamare          | Quando                                                                               |
|------------------------|--------------------------------------------------------------------------------------|
| tutto posizionale      | uno o due parametri, ordine ovvio: <code>calcola_iva(100.00)</code>                  |
| posizionale + nominato | i primi sono ovvi, gli ultimi no: <code>applica_sconto(89.00, percentuale=25)</code> |
| tutto nominato         | parametri dello stesso tipo che si possono confondere                                |
| sempre nominato        | valori booleani: <code>stampa_bolla(codice, zona, urgente=True)</code>               |

*Non c'è una regola unica, c'è un criterio: chi legge. Se la chiamata si capisce da sola i nomi non servono; se per sapere cosa significa quel numero bisogna aprire la definizione, servono. L'ultima riga vale la pena leggerla due volte: un True o un False in mezzo a una chiamata non dice niente a chi legge, `urgente=True` dice tutto.*

# 10. RESTITUIRE DUE VALORI

---

return multiplo e unpacking

## 10.1 return con la virgola: due valori restituiti insieme

```
def analizza(primo, secondo, terzo):  
    """Restituisce due cose: il totale e la media dei tre numeri."""  
    totale = primo + secondo + terzo  
    media = totale / 3  
  
    # Due nomi separati da virgola dopo return: escono insieme, in  
    # quest'ordine, e l'ordine e' una decisione di progetto.  
    return totale, media  
  
# UNPACKING: due nomi a sinistra dell'uguale, una chiamata sola a destra.  
# TRABOCCHETTO: i nomi a sinistra non devono chiamarsi come quelli dentro  
# la funzione. Qui coincidono per comodita' di lettura, ma sono  
# variabili diverse: quelle dentro erano locali e sono gia' sparite.  
totale, media = analizza(10, 20, 30)  
print(f"Totale: {totale}")  
print(f"Media: {media}")  
  
# Totale: 60  
# Media: 20.0
```

*Si chiama unpacking, e lo usate dal Giorno 03 senza saperlo: è la stessa cosa di `zona, peso = riga.split(";")`.*

## 10.2 L'ordine dell'unpacking: nessun errore se lo invertite

Scrivendo `media, totale = analizza(10, 20, 30)`

cioè invertendo i due nomi a sinistra:

`media` vale 60

`totale` vale 20.0

Nessun errore, e i due significati sono scambiati.

È lo stesso problema degli argomenti in ordine sbagliato, e stavolta

nessuna forma nominata vi salva: a sinistra dell'uguale i nomi non esistono.

C'è solo la docstring, che deve dire esplicitamente in che ordine escono le cose.

## 10.3 Quanti valori restituire: due sì, quattro è un segnale

```
def esamina_ticket(testo):  
    """Restituisce due cose: la categoria del ticket e il punteggio spam."""  
    # Due chiamate dentro un return solo: escono i due valori che le due  
    # funzioni hanno restituito, nell'ordine dichiarato dalla docstring.  
    return classifica_ticket(testo), punteggio_spam(testo)  
  
# TRABOCCHETTO: sei nomi a sinistra sono sei occasioni di sbagliare  
# ordine, e la docstring che li elenca tutti diventa illeggibile. Il  
# problema non e' la riga: e' che quei sei valori sono un'unica cosa  
# senza nome.  
codice, zona, peso, urgente, imponibile, iva = calcola_tutto(riga)  
  
# Due va benissimo, tre si tollera, quattro e' il segnale che vi serve  
# una struttura dati. Oggi fermatevi a due.
```

*Le due funzioni chiamate qui rispondono a due domande diverse sullo stesso testo e restano separate: `classifica_ticket()` dice di che cosa parla il ticket e restituisce una fra cinque categorie dell'Aurora, `punteggio_spam()` dice quanto pesano le parole spia. Un ticket può essere un RECLAMO legittimo e insieme prendere due punti per la parola rimborso: fonderle vorrebbe dire non poter più rispondere né all'una né all'altra.*

## 10.4 Senza unpacking: il pacchetto con le parentesi tonde

```
def analizza(primo, secondo, terzo):  
    totale = primo + secondo + terzo  
    return totale, totale / 3  
  
# TRABOCCHETTO: a sinistra c'e' UN nome solo e la funzione ne restituisce  
# due. Non e' un errore: i due valori restano impacchettati insieme, e  
# risultato contiene il pacchetto intero.  
risultato = analizza(10, 20, 30)  
  
# Le parentesi tonde nell'output non le ha scritte nessuno: sono il modo  
# in cui Python mostra quel pacchetto.  
print(risultato)  
  
# (60, 20.0)
```

*Quel pacchetto ha un nome e oggi non ci serve. Tenetevi la formulazione operativa: una funzione può restituire due cose, e chi la chiama le riceve mettendo due nomi separati da virgola a sinistra dell'uguale. Che cosa sia davvero quel pacchetto lo vedrete quando arriveranno le strutture dati.*

## 10.5 Da provare subito: minimo e massimo in un return solo

```
def minimo_e_massimo(a, b, c):  
    # min() e max() sono due funzioni già pronte di Python: le chiamate qui  
    # dentro sono composizione, esattamente come al capitolo 7.  
    return min(a, b, c), max(a, b, c)  
  
# TRABOCCHETTO: invertendo i due nomi a sinistra non cambia niente a  
# schermo tranne i numeri. Nessun errore vi avvisa dello scambio.  
piccolo, grande = minimo_e_massimo(7, 2, 9)  
print(piccolo, grande)  
  
# 2 9
```

*Prevedete l'output prima di eseguire. Poi invertite i due nomi a sinistra e rieseguite: scrivete in chat cosa cambia. Nessun errore in nessuno dei due casi, ed è il punto.*

# 11. AMBITO DELLE VARIABILI: LOCALE E GLOBALE

---

Quello che nasce dentro, muore dentro

# 11.1 Ambito locale: che cosa esce da una funzione

Analogia - il camerino.

Entrate in camerino con una maglietta, ve la provate, uscite.

Quello che è successo dentro resta dentro: fuori non avete la maglietta addosso, a meno che non l'abbiate comprata.

Il return è la cassa: è l'unico modo per portare fuori qualcosa.

Detta senza analogia: le variabili create dentro una funzione esistono solo mentre la funzione gira.

Non è un limite da aggirare: è la ragione per cui potete usare una funzione scritta da un altro senza doverla aprire.

## 11.2 La variabile locale sparisce: NameError fuori dalla funzione

```
def calcola_commissione(importo):  
    commissione = importo * 0.5 / 100  
    return commissione
```

```
calcola_commissione(2400.00)  
print(commissione)  
# La chiamata ha eseguito tutto: la commissione e' stata calcolata,  
# restituita e buttata via, perche' nessuno l'ha raccolta.  
# TRABOCCHETTO: commissione e' un nome nato DENTRO la funzione. Fuori non  
# e' "nascosto" o "vuoto": non esiste piu', ed e' per questo che  
# l'errore dice "is not defined" e non "has no value".
```

```
Traceback (most recent call last):  
  File "prova.py", line 6, in <module>  
    print(commissione)  
    ^^^^^^^^^^^  
NameError: name 'commissione' is not defined
```

*La forma corretta prende il valore dal return e lo salva fuori, con un nome che può anche essere diverso: spesa = calcola\_commissione(2400.00).*

## 11.3 Costanti globali: leggibili dentro, non riassegnabili

```
# Nome definito a colonna 0: e' globale, visibile da tutto il file.

totale = 100


def aggiungi():
    # TRABOCCHETTO: questa NON modifica il totale di sopra. Un'assegnazione
    #   dentro una funzione crea un nome locale nuovo, che oscura il globale
    #   per tutta la chiamata: due variabili distinte, stesso nome scritto.
    totale = 999
    print("dentro:", totale)


aggiungi()

# Il globale non e' mai stato toccato: vale ancora 100.
print("fuori:", totale)


# dentro: 999

# fuori: 100
```

*Dentro una funzione le costanti in cima al file si LEGGONO: è il motivo per cui ALIQUOTA\_IVA funziona dentro calcola\_iva senza essere passata come parametro. Ma se ASSEGNATE un valore a un nome, quel nome diventa locale, anche se fuori esisteva già.*

## 11.4 UnboundLocalError: assegnare a un nome che esisteva fuori

```
totale = 100
```

```
def aggiungi():  
    totale = totale + 1  
    print(totale)
```

```
aggiungi()
```

```
Traceback (most recent call last):  
  File "prova.py", line 9, in <module>  
    aggiungi()  
    ^^^^^^^^^  
  File "prova.py", line 5, in aggiungi  
    totale = totale + 1  
    ^^^^^
```

UnboundLocalError: cannot access local variable 'totale' where it is not associated with a value

*Sembra assurdo: totale esiste, vale 100, sta due righe sopra. Il punto è che Python, leggendo totale = ... dentro la funzione, ha deciso che totale è locale, e una variabile locale a quella riga non ha ancora nessun valore. La soluzione non è un trucco: aggiungi(valore) che fa return valore + 1, chiamata con totale = aggiungi(totale).*

## 11.5 La parola chiave global, e perché qui non si usa

Esiste una parola chiave che permette a una funzione di modificare una variabile di fuori. Si chiama global, e in questo corso non la usiamo.

Non è pigrizia didattica.

Il bisogno di global è quasi sempre il segnale che il programma va organizzato diversamente: se una funzione deve cambiare qualcosa che sta fuori, quella cosa si passa come argomento e si restituisce con return.

In un modulo dove dieci funzioni scrivono sulle stesse tre variabili globali, quando un numero è sbagliato non si sa chi lo ha scritto.

Se la conoscete e la usate nei vostri esercizi, nessun problema: nelle soluzioni ufficiali non c'è.

## 11.6 Il vantaggio dell'ambito: nomi locali che non si scontrano

Sembra un vincolo, è una libertà.

In 06.6 scriverete due funzioni che usano tutte e due una variabile chiamata contatore, e non si disturbano:  
sono due contatore diversi, ognuno vive nella sua funzione.

Se le variabili locali non esistessero, chi scrive la seconda dovrebbe sapere quali nomi ha usato chi ha scritto la prima.

Ingestibile in un modulo di trenta funzioni.  
Impossibile in un programma scritto da tre persone.

# 12. NOMI DI FUNZIONE E DI PARAMETRO

---

La prima documentazione: verbo più oggetto

## 12.1 Nomi di funzione: verbo più oggetto

| Nome              | Giudizio                                            |
|-------------------|-----------------------------------------------------|
| calcola_totale    | buono: dice cosa fa e su cosa                       |
| classifica_ticket | buono                                               |
| valida_cap        | buono                                               |
| stampa_bolla      | buono, e dice anche che stampa invece di restituire |
| dati              | inutile: quali dati, e cosa ci fa?                  |
| funzione2         | dannoso                                             |
| elabora           | elabora cosa, e come?                               |
| check             | mezzo inglese, e non dice cosa controlla            |

*Analogia - il nome sulla porta. In un ufficio le targhette dicono "Ufficio protocollo", "Sala server": sapete dove andare senza aprire. Una porta con scritto "Stanza 2" vi costringe a girarle tutte, e una porta con scritto "Archivio" che dentro ha la caldaia è peggio di una senza targhetta, perché chi legge si fida. Chi cerca dove si calcola lo sconto in un modulo di trenta funzioni non le legge: scorre i nomi.*

## 12.2 Il prefisso dichiara la forma: calcola\_, stampa\_, chiedi\_

| Prefisso                                    | Cosa aspettarsi                                        |
|---------------------------------------------|--------------------------------------------------------|
| calcola_, conta_, costo_, prezzo_           | restituisce un numero                                  |
| classifica_, verdetto_, nome_, descrizione_ | restituisce un testo                                   |
| e_, ha_, valida_                            | restituisce True o False                               |
| stampa_, mostra_                            | stampa e non restituisce niente                        |
| chiedi_                                     | chiede all'utente e restituisce quello che ha ottenuto |

*Un nome ben scelto vi dice anche se la funzione restituisce o stampa. Tenete questa tabella sott'occhio mentre fate 06.8: le sei funzioni del mini-progetto usano tutte uno di questi prefissi, e non è un caso.*

## 12.3 La regola della "e": un nome con la congiunzione sono due funzioni

```
# 1. Chi calcola non stampa: restituisce il numero grezzo, senza decidere
#     come va mostrato. E' la funzione riusabile delle due.
def calcola_iva(imponibile):
    """Restituisce l'IVA dovuta su un imponibile."""
    return imponibile * 0.22

# 2. Chi stampa non calcola: chiede il numero a chi lo sa fare e si occupa
#     solo della presentazione, etichetta e due decimali compresi.
def stampa_iva(imponibile):
    """Stampa l'IVA dovuta. Non restituisce niente."""
    print(f"IVA: {calcola_iva(imponibile):.2f}")

# TRABOCCHETTO: le due righe qui sotto scrivono quasi lo stesso numero, ma
#     solo la seconda vi lascia in mano un valore da sommare.
stampa_iva(100.00)
print(f"{calcola_iva(100.00):.2f}")

# IVA: 22.00
# 22.00
```

## 12.4 Nomi dei parametri: la firma che si legge senza aprire il corpo

`def calcola(a, b):`

`def applica_sconto(prezzo, percentuale):`

Possono avere lo stesso identico corpo.

Il primo richiede di leggerlo per capire cosa vuole.

Il secondo si usa senza aprirlo.

E c'è un secondo guadagno: con i nomi buoni funzionano anche gli argomenti nominati del capitolo 9, che con `a` e `b` non servirebbero a niente.

`applica_sconto(prezzo=89.00, percentuale=25)` si legge.

`calcola(a=89.00, b=25)` no.

# 13. LA DOCSTRING

---

Tre righe che fra sei mesi vi salvano

## 13.1 La docstring: prima riga del corpo, fra tripli apici

```
def calcola_spedizione(zona, peso, urgente=False):  
    """Restituisce due valori: l'imponibile e l'IVA di una spedizione.  
  
    Riceve la zona in maiuscolo, il peso in chili e un booleano per  
    l'urgenza. Non stampa niente e non chiede niente all'utente.  
    """  
  
    # TRABOCCHETTO: la docstring deve essere la PRIMA istruzione del corpo.  
    # Spostatela sotto la riga che segue e diventa una stringa in mezzo al  
    # codice, che Python valuta e butta: nessun errore, niente documentato.  
    imponibile = 4.50 + peso * 0.80  
  
    if urgente:  
        imponibile = imponibile + imponibile * 30 / 100  
  
    # Due valori restituiti: l'ordine e' quello annunciato dalla prima riga  
    # della docstring, ed e' l'unico posto in cui sta scritto.  
    return imponibile, imponibile * 0.22  
  
  
    # __doc__ e' il nome sotto cui Python tiene la docstring attaccata alla  
    # funzione: la prova che non e' stata buttata come un commento.  
  
    print(calcola_spedizione.__doc__)
```

*Non è un commento: Python la conserva, e quel print ne stampa il testo, mentre un commento con il cancelletto sparisce. In PyCharm la docstring è quello che compare passando il puntatore sul nome della funzione, o con `Ctrl+Q` (`F1` su Mac).*

## 13.2 Contenuto della docstring: cosa fa, cosa riceve, cosa restituisce

Tre informazioni, in quest'ordine.

1. COSA FA, in una riga che comincia con un verbo.
2. COSA RICEVE, se non è ovvio dai nomi dei parametri.
3. COSA RESTITUISCE, sempre.

Soprattutto se restituisce due cose: l'ordine in cui escono non sta scritto in nessun altro posto del programma.

Per le funzioni corte basta la prima riga.

## 13.2 Per le funzioni corte basta la prima riga

```
def kg_in_libbre(kg):  
    """Restituisce il peso in libbre corrispondente a un peso in chili."""  
    # Una riga sola basta perche' dice le tre cose insieme: che restituisce,  
    # che cosa, e con quale unita' di misura in ingresso e in uscita.  
    return kg * 2.20462  
  
print(f"{kg_in_libbre(12.0):.2f}")  
  
# 26.46
```

*Una funzione di due righe con quattro righe di docstring è sbilanciata: la documentazione deve costare meno del codice che descrive, o smetterete di scriverla.*

## 13.3 Formato della docstring: cinque regole di forma

| Regola                                                 | Perché                                         |
|--------------------------------------------------------|------------------------------------------------|
| triplici apici, anche per una riga sola                | è la convenzione, e permette di allungarla poi |
| comincia con la maiuscola e finisce con il punto       | è una frase                                    |
| verbo all'inizio: Restituisce..., Stampa..., Chiede... | dice subito la forma della funzione            |
| se restituisce due valori, dirlo e dire in che ordine  | non c'è nessun altro modo di saperlo           |
| non ripetere il nome della funzione                    | "Calcola l'IVA" su calcola_iva è aria fritta   |

*Sembrano pignolerie e non lo sono: una docstring scritta sempre allo stesso modo si legge in due secondi, una scritta ogni volta diversa va letta tutta.*

## 13.4 La docstring dichiara anche i limiti

```
def conta_vocali(testo):  
    """Conta le vocali di un testo, senza distinguere maiuscole e minuscole.  
  
    Le vocali accentate (à, è, ì, ò, ù) non vengono contate.  
    """  
    vocali = 0  
    for carattere in testo.lower():  
        # TRABOCCHETTO: la stringa "aeiou" contiene cinque caratteri e nessuno di  
        # loro e' accentato. Non e' un difetto nascosto: e' una scelta, e la  
        # seconda riga della docstring la dichiara a chi usera' la funzione.  
        if carattere in "aeiou":  
            vocali += 1  
    return vocali  
  
print(conta_vocali("città"))
```

# 2

*Su "città" questa funzione risponde 2, non 3. Chi legge la docstring lo sa prima di scoprirlo su un catalogo di cinquemila prodotti: è il posto giusto per essere onesti su quello che la funzione non fa.*

# 14. RIFATTORIZZAZIONE

---

Estrarre funzioni a comportamento invariato

## 14.1 Il metodo di rifattorizzazione, in cinque passi

| Passo | Cosa si fa                                                 | Come si verifica                                  |
|-------|------------------------------------------------------------|---------------------------------------------------|
| 1     | Eseguire la versione vecchia e salvare l'output            | copiarlo in un file di testo o in chat            |
| 2     | Cercare i blocchi che hanno già un nome nella vostra testa | scriverli su un foglio prima di toccare il codice |
| 3     | Estrarre una funzione per volta                            | mai due insieme                                   |
| 4     | Rieseguire e confrontare con l'output salvato              | carattere per carattere                           |
| 5     | Solo se è identico, passare alla successiva                | altrimenti si torna indietro                      |

*La regola d'oro è una: il comportamento osservabile non deve cambiare. Il passo 3 è quello che si salta sempre ed è quello che fa la differenza: se estraete quattro funzioni insieme e l'output cambia, non sapete quale delle quattro ha rotto qualcosa.*

## 14.2 PRIMA: due copie divergenti dello stesso listino

```
def main():  
    """Calcola lo scontrino di due articoli."""  
  
    totale = 0.0  
  
    # 1. Primo articolo: il listino scritto come catena di if.  
  
    codice = "MON"  
  
    if codice == "CUF":  
        totale += 49.90  
  
    elif codice == "MON":  
        totale += 219.00  
  
    # TRABOCCHETTO: il secondo articolo riusa la variabile codice e ricopia  
    # la catena con un ramo diverso. Sono due copie divergenti dello stesso  
    # listino: CUF c'e' in tutte e due, MON e MOU in una sola.  
  
    codice = "MOU"  
  
    if codice == "CUF":  
        totale += 49.90  
  
    elif codice == "MOU":  
        totale += 34.50
```

*Il programma funziona e l'output è corretto: il difetto non è nel risultato, è nella forma. Prima di toccare qualunque cosa si esegue e si salva l'output.*

## 14.2 PRIMA: lo sconto e la stampa, incollati sotto

```
# 2. Secondo lavoro, incollato sotto il primo: dal totale alla
#    percentuale. Stesso rientro, argomento diverso, nessun segno di
#    confine: niente dice che qui si cambia mestiere.

if totale >= 200.00:
    percentuale = 10
elif totale >= 100.00:
    percentuale = 5
else:
    percentuale = 0

# 3. Terzo lavoro: la presentazione. Il calcolo dello scontato e' scritto
#    dentro la f-string, cioe' nel punto meno visibile del programma.

print(f"Totale lordo: {totale:.2f}")

print(f"Da pagare: {totale - totale * percentuale / 100:.2f}")


main()


# Totale lordo: 253.50
# Da pagare: 228.15
```

*Diciotto righe dentro main(), tre mestieri diversi e nessuna etichetta. Quei due numeri di output sono il metro di collaudo: da qui in avanti non devono cambiare di un carattere.*

## 14.2 Primo passo: si estrae prezzo\_articolo(), e solo quella

```
def prezzo_articolo(codice):  
    """Restituisce il prezzo di listino di un codice, 0.0 se non esiste."""  
  
    # 1. Valore di partenza: il codice sconosciuto vale 0.0, e la docstring  
    #    lo dichiara. Senza questa riga la funzione restituirebbe None (6.4).  
    prezzo = 0.0  
  
    # 2. Le due catene divergenti di prima diventano una catena sola e  
    #    completa: l'unificazione e' il guadagno vero dell'estrazione.  
    if codice == "CUF":  
        prezzo = 49.90  
    elif codice == "MOU":  
        prezzo = 34.50  
    elif codice == "MON":  
        prezzo = 219.00  
    return prezzo  
  
# Dentro main() le prime dieci righe diventano due:  
#     totale = prezzo_articolo("MON") + prezzo_articolo("MOU")
```

*Si esegue: l'output è ancora Totale lordo: 253.50 e Da pagare: 228.15. Identico, quindi si va avanti. Se fosse cambiato si tornerebbe indietro, e il pezzo da guardare sarebbe uno solo.*

## 14.2 Secondo passo: percentuale\_sconto() e le soglie con un nome

```
# Le quattro costanti in cima: soglie e percentuali smettono di essere

# numeri sparsi dentro un if e diventano parametri del programma.

SOGLIA_SCONTO_ALTO = 200.00

SCONTO_ALTO = 10

SOGLIA_SCONTO_BASSO = 100.00

SCONTO_BASSO = 5


def percentuale_sconto(totale):

    """Restituisce la percentuale di sconto che spetta a un totale."""

    percentuale = 0

    # TRABOCCHETTO: le soglie vanno confrontate in ordine DECRESCENTE. Con il
    # 100.00 scritto per primo, un totale di 253.50 si fermerebbe al primo
    # ramo vero e prenderebbe il 5% invece del 10%, senza nessun errore.

    if totale >= SOGLIA_SCONTO_ALTO:

        percentuale = SCONTO_ALTO

    elif totale >= SOGLIA_SCONTO_BASSO:

        percentuale = SCONTO_BASSO

    return percentuale
```

*La soglia dei duecento euro, quella che a Giulia era costata quattro minuti, adesso sta in una costante in cima al file ed è la prima cosa che si vede aprendolo. La richiesta di stamattina è diventata una riga sola.*

## 14.2 Il risultato: main() diventa il regista

```
def main():  
    """Calcola lo scontrino di due articoli."""  
  
    # main() adesso e' il regista: quattro righe, nessun prezzo, nessuna  
    # soglia. Ogni riga nomina un passaggio.  
  
    totale = prezzo_articolo("MON") + prezzo_articolo("MOU")  
  
    percentuale = percentuale_sconto(totale)  
  
    print(f"Totale lordo: {totale:.2f}")  
  
    print(f"Da pagare: {totale - totale * percentuale / 100:.2f}")  
  
  
main()  
  
  
# Totale lordo: 253.50      <- identico  
  
# Da pagare: 228.15       <- identico
```

*main() è passato da diciotto righe a quattro, e adesso si legge come una frase: il totale è la somma dei due prezzi, la percentuale dipende dal totale, si stampa. Il controllo veloce da fare su 06.4 è uno solo: main() contiene tariffe, prezzi o soglie? Se sì, il lavoro non è finito.*

## 14.2 Lo stesso programma, prima e dopo

PRIMA

```
-----  
def main():  
    totale = 0.0  
    codice = "MON"  
    if codice == "CUF":  
        totale += 49.90  
    elif codice == "MON":  
        totale += 219.00  
    codice = "MOU"  
    if codice == "CUF":  
        totale += 49.90  
    elif codice == "MOU":  
        totale += 34.50  
    if totale >= 200.00:  
        percentuale = 10  
    ...  
-----
```

un pezzo solo: per cambiare la soglia  
bisogna prima trovarla

*Stesso programma, stesso output, stesso comportamento riga per riga. A sinistra si cerca dentro come in un cassetto; a destra si legge un elenco di passaggi, e ogni passaggio ha un nome. È l'esercizio 06.4, sulla cassa vera da sei articoli.*

DOPO

```
-----  
SOGLIA_SCONTO_ALTO = 200.00  
SCONTO_ALTO = 10  
...  
  
def prezzo_articolo(codice):  
    ...restituisce 49.90 / 34.50 / 219.00  
  
def percentuale_sconto(totale):  
    ...restituisce 10, 5 oppure 0  
  
def main():  
    totale = prezzo_articolo("MON") + ...  
    percentuale = percentuale_sconto(...)  
    print(...)  
-----
```

tre pezzi con un nome e un regista:  
la soglia e' una costante in cima

## 14.3 Cinque errori di rifattorizzazione e i loro sintomi

| Errore                                        | Sintomo                          | Rimedio                       |
|-----------------------------------------------|----------------------------------|-------------------------------|
| estrarre troppe funzioni insieme              | l'output cambia e non si sa dove | una per volta                 |
| estrarre una funzione da sessanta righe       | il nome ha una "e" dentro        | spezzarla in due              |
| lasciare una print in una funzione di calcolo | output doppio                    | chi calcola non stampa        |
| dimenticare il return                         | None a valle                     | eseguire dopo ogni estrazione |
| cambiare un numero "già che ci siamo"         | l'output cambia legittimamente   | sono due lavori diversi       |

*L'ultimo merita un'insistenza: durante una rifattorizzazione non si correggono bug e non si aggiungono funzionalità. Prima si riorganizza a comportamento invariato e si verifica, poi si corregge. Se mescolate le due cose e l'output cambia, non sapete quale delle due lo ha cambiato.*

## 14.4 Dimensione di una funzione: due sintomi affidabili

Non esiste una misura. Esistono due sintomi affidabili.

1. Se per capire cosa fa una funzione dovete scorrere, è troppo grande.
2. Se ha una riga sola e viene chiamata una volta sola, probabilmente non serviva.

Nel mezzo c'è tutto lo spazio ragionevole:

le funzioni delle soluzioni di oggi stanno fra tre e venti righe.

In 06.4 metà aula estrarrà una funzione da sessanta righe.

È l'occasione per la regola della "e": se il nome ha una "e" dentro, sono due funzioni.

# 15. ERRORI TIPICI DELLE FUNZIONI

---

Quattro che si vedono, quattro che costano

# E1 - Il return dimenticato: TypeError su NoneType

```
def calcola_iva(imponibile):  
    imponibile * 0.22          # manca il return
```

```
Traceback (most recent call last):  
  File "prova.py", line 5, in <module>  
    totale = 100.00 + calcola_iva(100.00)  
                ~~~~~^~~~~~  
TypeError: unsupported operand type(s) for +: 'float' and 'NoneType'
```

La moltiplicazione c'è, il risultato viene calcolato e buttato.  
CORREZIONE: aggiungere return davanti all'espressione.

*È il numero uno con distacco, e si presenta lontano dalla causa. Il riflesso da costruire oggi: se leggete NoneType, cercate una funzione senza return.*

## E2 - La chiamata prima della definizione: NameError

```
print(prezzo_articolo("CUF"))          # scritto SOPRA il def
```

```
Traceback (most recent call last):  
  File "prova.py", line 1, in <module>  
    print(prezzo_articolo("CUF"))  
    ^^^^^^^^^^^^^^^^^^^
```

```
NameError: name 'prezzo_articolo' is not defined
```

CORREZIONE: funzioni in alto, main() sotto, chiamata in fondo.

*Il nome nasce nell'istante in cui Python legge il def. Se la chiamata sta sopra, in quell'istante il nome non esiste ancora: non è un problema di sintassi, è un problema di ordine.*

## E3 - L'argomento mancante: missing 1 required positional argument

```
print(appllica_sconto(89.00))      # la funzione vuole due parametri
```

```
Traceback (most recent call last):
```

```
  File "prova.py", line 5, in <module>
```

```
    print(appllica_sconto(89.00))
```

```
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
TypeError: appllica_sconto() missing 1 required positional argument:  
'percentuale'
```

CORREZIONE: passare il secondo argomento, oppure - se ha senso -  
dargli un valore di default.

*Python vi dice anche come si chiama il parametro mancante: è il messaggio d'errore più collaborativo della giornata, e va letto fino in fondo invece che a colpo d'occhio.*

## E4 - La variabile locale usata fuori: NameError sul nome interno

```
calcola_iva(100.00)
print(iva)                # iva e' nata dentro la funzione
```

```
Traceback (most recent call last):
  File "prova.py", line 7, in <module>
    print(iva)
      ^^^
```

NameError: name 'iva' is not defined

CORREZIONE: salvare il valore restituito.

```
iva = calcola_iva(100.00)
```

*Stesso messaggio di E2, causa opposta: lì mancava una definizione di funzione, qui manca la raccolta di un valore di ritorno. Il nome che compare nel messaggio dice quale dei due casi avete davanti.*

## E5 - La funzione chiamata senza parentesi: nessun errore

```
def saldo_disponibile(saldo, fido):  
    return saldo + fido
```

```
# TRABOCCHETTO: mancano le parentesi e mancano i due argomenti, e nessuno  
# protesta. Il nome nudo vale la funzione stessa: disponibile diventa  
# un secondo nome per quella funzione, non il suo risultato.  
disponibile = saldo_disponibile  
print("Disponibile:", disponibile)
```

```
# Disponibile: <function saldo_disponibile at 0x0000020F1C2E5620>
```

```
print(type(disponibile))
```

```
# <class 'function'>
```

## E6 - Gli argomenti nell'ordine sbagliato: risultato assurdo, zero errori

Visto al paragrafo 4.3.

```
rata_mensile(12, 2400.00) restituisce 0.005  
invece di 200.0
```

Nessun errore. Numero assurdo.

Nessun modo, per Python, di accorgersene: due numeri sono due numeri, e la divisione si può fare in tutti e due i versi.

**CORREZIONE E PREVENZIONE:** argomenti nominati.

```
rata_mensile(capitale=2400.00, mesi=12)
```

## E7 - Il return dentro il ciclo: la funzione esce al primo giro

Visto al paragrafo 5.3.

`totale_rate(200.00, 12)` restituisce 200.0  
invece di 2400.0

Il return è indentato dentro il for, e la funzione esce al primo giro.

Nessun errore.

CORREZIONE: quattro spazi a sinistra sul return,  
non un ritocco al ciclo. Il ciclo era giusto.

In aula virtuale quattro spazi sono invisibili: questo va fatto  
sperimentare a schermo condiviso, non raccontato.

## E8 - Il dato malformato e il limite della validazione preventiva

Un errore che oggi non sappiamo ancora chiudere come si deve.

`chiedi_intero()` del paragrafo 5.4 valida con `if` e `.isdecimal()`,  
e regge finché il dato lo digita una persona.

Quando arriva da un file mandato dal fornitore, e nella colonna del peso  
c'è scritto venti invece di 20, la validazione preventiva diventa faticosa:  
le forme sbagliate vanno previste tutte, e una sfugge sempre.

Esiste uno strumento apposito per raccogliere l'errore dopo che è successo  
invece di prevenirlo: si chiama `try/except`, e non è materia di queste  
undici giornate.

Qui ci fermiamo alla validazione preventiva: resta la prima linea di difesa.

# E1-E8 Tabella di pronto soccorso

| Messaggio o sintomo                         | Cosa è successo                  | Prima cosa da guardare          |
|---------------------------------------------|----------------------------------|---------------------------------|
| TypeError: ... 'NoneType'                   | manca il return                  | la funzione appena chiamata     |
| NameError su un nome di funzione            | chiamata sopra il def            | l'ordine nel file               |
| NameError su un nome di variabile           | variabile locale usata fuori     | salvare il valore restituito    |
| TypeError: missing 1 required positional... | argomento mancante               | la chiamata                     |
| SyntaxError: parameter without a default... | default prima di un obbligatorio | l'ordine dei parametri          |
| SyntaxError: positional argument follows... | posizionale dopo un nominato     | l'ordine nella chiamata         |
| UnboundLocalError                           | assegnazione a un nome di fuori  | passare il valore e restituirlo |
| Il totale è 200 invece di 2400              | return dentro il ciclo           | i quattro spazi del return      |
| Esce <function ... at 0x...>                | chiamata senza parentesi         | le parentesi                    |

*I primi sei danno un traceback e si vedono. Gli ultimi tre no, e sono quelli che costano: il programma gira, non protesta, e il numero è sbagliato.*

# Riepilogo del giorno (1/2)

Perché: un programma lungo non si trova, non si cambia e non si riusa.

Tre segnali: serve un commento, state ricopiando, dovete risalire di quaranta righe.

def nome(parametri): e il corpo indentato. Definire non è eseguire.

Funzioni in alto, main() sotto, la chiamata in fondo dentro la guard.

Due righe vuote fra una funzione e l'altra, una per separare i pensieri.

Il parametro sta nella definizione, l'argomento nella chiamata.

return restituisce, print mostra: chi calcola non stampa, chi stampa non calcola.

return interrompe la funzione, anche al primo giro di un ciclo.

Senza return una funzione restituisce None: il sintomo è TypeError ... NoneType,  
e compare lontano dalla causa. Un valore di partenza, gli if, un return solo.

# Riepilogo del giorno (2/2)

Una funzione ne può chiamare un'altra: cinque righe che ne chiamano tre si leggono come una frase. Il traceback con più righe File è la mappa del percorso.

Default: dopo i parametri obbligatori, e solo quando esiste un valore giusto, non un valore comodo.

Argomenti nominati: l'ordine non conta più, i booleani si passano sempre nominati.

return totale, media e poi totale, media = analizza(...): l'ordine lo dice la docstring. Quello che nasce dentro muore dentro: una costante si legge, non si riassegna. global esiste e in questo corso non si usa.

Verbo più oggetto, e se il nome ha una "e" dentro sono due funzioni.

Rifattorizzare: una funzione per volta, si riesegue, l'output non deve cambiare.

# Grazie.

---

*shaping the skills of tomorrow*

